

Auditoría Completa — Backend-Travel-Agency-2

Fecha: 16 de mayo de 2026 **Proyecto:** Backend-Travel-Agency-2 (Spring Boot + Maven + MySQL) **Stack real:** Java 25 + Spring Boot 4.0.6 + Maven + MySQL

1. Resumen Ejecutivo

API RESTful para una agencia de viajes. Proyecto bien estructurado en capas (controller → service → repository → entity) con manejo de excepciones centralizado, DTOs, mappers, motor de precios robusto, seguridad JWT, Cloudinary, y email transaccional.

Puntuación general: 8.5/10

La mayoría de los issues críticos detectados en auditorías previas han sido corregidos. El principal problema actual son **tests desactualizados** que no compilan contra el código refactorizado.

2. Stack Tecnológico

Tecnología	Versión
Java	25
Spring Boot	4.0.6
Spring Data JPA	Hibernate
MySQL	8+ (mysql-connector-j)
Maven	3.9.14 (wrapper)
Lombok	Última (via Spring Boot)
Cloudinary	1.39.0
Auth0 java-jwt	4.4.0
jBCrypt	0.4
SpringDoc OpenAPI	2.8.5
Spring Boot Mail	via parent
Spring Boot Thymeleaf	via parent
Tests	JUnit 5 + Mockito + H2

3. Estructura del Proyecto

```
src/main/java/com/inditex/g1_agencia_viajes/  
├─ G1AgenciaViajesApplication.java      # Entry point  
├─ config/                             # CloudinaryConfig, CorsConfig, AsyncConfig  
├─ controller/                         # 12 controladores REST
```

─ AuthenticationController.java	# POST /api/authentication/login
─ BookingController.java	# /api/bookings
─ BusController.java	# /api/buses
─ CloudinaryController.java	# /api/images
─ DashboardController.java	# /api/dashboard
─ DriverController.java	# /api/drivers
─ EmployeeController.java	# /api/employees
─ HotelController.java	# /api/hotels
─ OfferController.java	# /api/offers
─ TravelController.java	# /api/travels
─ TripSegmentController.java	# /api/trip-segments
─ UserController.java	# /api/users
─ dto/	# 24 DTOs de request/response
─ exception/	# 10 excepciones + GlobalExceptionHandler
─ mapper/	# 9 mappers
─ model/	# 9 entidades + 3 enums
─ repository/	# 9 repositorios JPA
─ security/	# JwtUtil, JwtFilter, SecurityConfig
─ service/	# 13 servicios

4. Modelo de Datos (BD)

Tabla	Propósito	FK
users	Clientes del sistema	tutor_id → users
employees	Empleados/agentes	—
hotels	Hoteles disponibles	—
buses	Autobuses	—
drivers	Conductores	—
offers	Ofertas temporales	—
travels	Viajes	hotel_id → hotels, offer_id → offers
trip_segments	Trayectos del viaje	travel_id → travels, bus_id → buses, driver_id → drivers
bookings	Reservas	travel_id → travels, employee_id → employees
customers_bookings	N:N bookings ↔ users	booking_id, customer_id

5. Escala de Valoración

- **Crítico:** Impide el funcionamiento o es un riesgo de seguridad grave. Arreglar inmediatamente.
- **Alto:** Impacta negativamente en calidad, mantenibilidad o funcionalidad. Arreglar pronto.
- **Medio:** Incumple buenas prácticas o convenios del proyecto. Mejorar cuando se pueda.
- **Punto fuerte:** Aspecto positivo del proyecto.

6. Issues por Categoría

6.1 Seguridad — ● 9/10

Aspecto	Estado	Detalle
JWT secret externalizado	✓	@Value("\${jwt.secret}") desde application.properties, sin fallback hardcodeado
Tokens con expiración	✓	24h configurable via jwt.expiration
JwtFilter autentica	✓	Verifica Bearer token + issuer + firma HMAC256
Roles implementados	✓	VIEWER (solo GET), EDITOR (no /api/employees), ADMIN (todo)
Rutas públicas limitadas	✓	Solo /api/authentication/login, /api-docs, /swagger-ui
BCrypt en contraseñas	✓	Seed data con hashes BCrypt válidos
Constructor injection	✓	Sin @Autowired en campos
Posible mismatch ruta Swagger	●	path.startsWith("/swagger-ui") en JwtFilter — debería funcionar para /swagger-ui.html y /swagger-ui/*

6.2 DTOs y Validaciones — ● 9/10

Archivo	Validaciones presentes
HotelRequestDTO.java	✓ @NotBlank en nombre/dirección/ciudad/país, @Min(1) @Max(5) en stars, @NotNull @Min(1) en capacity
BookingRequestDTO.java	✓ @NotNull typeBoard/travelId, @NotEmpty customerId
BusRequestDTO.java	✓ @NotBlank + @Pattern("[0-9]{4}-[A-Z]{3}") en licensePlate, @NotNull @Min(1) capacity
DriverRequestDTO.java	✓ @NotBlank + @Pattern en phone (^\\+?[0-9]{7,15}\$)
UserRequestDTO.java	✓ @NotBlank + @Email, @Pattern en dni ([0-9]{8}[A-Z]\$) y passport ([A-Z]{3}[0-9]{6}\$)
EmployeeController.java	✓ Usa DTOs (EmployeeRequestDTO/EmployeeResponseDTO) + @Valid en todos los @RequestBody

6.3 Manejo Global de Excepciones — ● 9/10

Handlers implementados (12):

Excepción	Código HTTP
MethodArgumentNotValidException	400
ResourceNotFoundException	404
HotelNotAvailableException	409

TravelNotAvailableException	409
EmailAlreadyExistsException	409
IllegalArgumentException	400
MinorWithoutTutorException	400
DuplicateLicensePlateException	409
DriverOverlapException	409
PastTravelException	409
BusFullException	409
HttpMessageNotReadableException	400
MethodArgumentTypeMismatchException	400
Exception (catch-all)	500

6.4 Reglas de Negocio — 9/10

Regla	Estado	Implementación
No vender viajes pasados	✓	BookingService.save() lanza PastTravelException si startDate <= LocalDate.now()
No vender si bus completo	✓	BookingService.save() cuenta pasajeros totales y compara con Bus.capacity
No vender si hotel completo	✓	HotelService.reduceCapacity() lanza HotelNotAvailableException
Menor acompañado de adulto	✓	validateMinorHasTutor() chequea edad < 18 y tutorId != null
Conductor no puede conducir 2 buses	✓	TripSegmentService usa findOverlappingByDriver() JPQL query
Tarifa niño (≤17, 15% desc.)	✓	BookingPricingService.determinePassengerDiscount()
Tarifa pensionista (≥65, 10% desc.)	✓	BookingPricingService.determinePassengerDiscount()
Descuento grupo (5%, ≥10 pax)	✓	BookingPricingService.buildQuote() aplica si isGroup && size >= 10
Descuento oferta	✓	BookingPricingService.resolveOfferDiscount() aplica % de offer asociado

6.5 Testing — 3/10

Problema crítico: Tests desactualizados tras refactorización a Pageable

Los servicios se refactorizaron para usar `Pageable` en métodos `getAll()`, `getAvailable()`, `getOnSale()`, pero los tests no se actualizaron. **Los tests no compilan.**

Archivo	Línea	Problema
TravelServiceTest.java	107, 117, 127	Llama a travelService.getAll() sin Pageable
HotelServiceTest.java	102, 132, 142	Llama a hotelService.getAll() / getActive() / getAvailable() sin Pageable
TripSegmentServiceTest.java	85	Llama a tripSegmentService.getAll() sin Pageable
UserServiceTest.java	131	Llama a userService.getAll() sin Pageable
BusServiceImplTest.java	74	Llama a busService.getAll() sin Pageable
DriverServiceTest.java	79	Llama a driverService.getAll() sin Pageable
TravelControllerTest.java	48, 57, 68	Mocks de travelService.getAll/getAvailable/getOnSale sin Pageable
HotelControllerTest.java	48, 86	Mocks de hotelService.getAll/getAvailable sin Pageable
UserControllerTest.java	48	Mock de userService.getAll() sin Pageable
DriverControllerTest.java	48	Mock de driverService.getAll() sin Pageable
BusControllerTest.java	48	Mock de busService.getAll() sin Pageable
TripSegmentControllerTest.java	48	Mock de tripSegmentService.getAll() sin Pageable

Tests que funcionarían (no afectados):

Clase	Tests	Cobertura
BookingPricingServiceTest	11	Precios, descuentos, quote
BookingServiceTest	16	CRUD, capacidad, menores, quote
EmployeeServiceTest	4	CRUD, password encryptado
OfferServiceTest	7	CRUD
AuthenticationControllerTest	—	Sin test
DashboardControllerTest	—	Sin test
CloudinaryControllerTest	—	Sin test

Tests faltantes adicionales:

- Repository tests (@DataJpaTest)
- CloudinaryService tests
- JwtFilter / SecurityConfig tests
- Tests de integración

6.6 Swagger/OpenAPI — ● 9/10

Aspecto	Estado
---------	--------

Dependencia springdoc-openapi	✓ Presente en pom.xml
Configuración	✓ /swagger-ui.html, /api-docs habilitados
@Tag en controllers	✓ Los 11 controllers tienen @Tag
@Operation en endpoints	✓ Todos los endpoints tienen @Operation
@ApiResponse	✗ No se usan en ningún controller
@Schema en DTOs	✗ No se usa

6.7 Convenciones REST — 7/10

Bien:

- Sustantivos en plural: /api/users , /api/hotels , /api/buses
- Métodos HTTP correctos (GET/POST/PUT/DELETE)
- Códigos de estado HTTP apropiados (201 → create, 200 → ok, 204 → delete)

Inconsistencias:

- README muestra /activos , /disponibles (español) pero el código usa /active , /available (inglés)
- Sin versionado de API (/api/v1/)
- /api/images/upload — estándar sería POST /api/images
- Nombres de métodos en controllers: getAllBookings vs getAll

6.8 Cloudinary — 9/10

Aspecto	Estado
Config externalizada	✓ CloudinaryConfig.java con @Value desde application.properties
Upload funcional	✓ CloudinaryService.uploadImage()
Delete funcional	✓ CloudinaryService.deleteImage()
Controller con Swagger	✓ @Tag + @Operation
Manejo errores específicos	✗ No captura timeouts ni archivos demasiado grandes

7. Issues Detallados

● C1 — Tests desactualizados (no compilan)

Archivos: 12+ archivos de test **Severidad:** Crítica — el proyecto no puede ejecutar tests **Causa:** Los servicios se refactorizaron para usar Pageable pero los tests mantienen firmas antiguas sin Pageable **Solución:** Actualizar todos los tests para usar Pageable en los mocks y llamadas a getAll(), getAvailable(), getOnSale()

● H1 — README endpoint names inconsistentes

Archivo: README.md **Severidad:** Baja — confunde al desarrollador **Detalle:** La tabla de endpoints muestra /activos , /disponibles pero los controllers reales usan /active , /available **Solución:** Actualizar README para reflejar los endpoints reales

● H2 — Operador + sobrante

Archivo: BookingPricingService.java:41 **Severidad:** Muy baja — no causa error funcional **Detalle:** "el viaje", + request.getTravelId() — el unario + es innecesario **Solución:** Eliminar el +

● H3 — data.sql con IDs fijos

Archivo: data.sql **Severidad:** Media — puede causar conflictos de IDs **Detalle:** Los INSERT usan IDs explícitos que asumen auto_increment empieza en 1. Si hay datos previos, fallará. **Solución:** Usar IDs relativos o limpiar la lógica de seed

● H4 — Sin @ApiResponse ni @Schema

Archivos: Todos los controllers **Severidad:** Baja — la documentación Swagger es funcional pero mejorable **Detalle:** Aunque hay @Tag y @Operation, faltan @ApiResponse para documentar errores y @Schema para DTOs **Solución:** Añadir @ApiResponse y @Schema a todos los endpoints

● M1 — Naming de endpoints inconsistente

Archivo: Todos los controllers **Severidad:** Baja **Detalle:** Mezcla de estilos: /available y /sale (inglés) vs naming inconsistente en métodos **Solución:** Unificar criterio de naming

● M2 — Faltan tests de controller

Archivos: DashboardController, AuthenticationController **Severidad:** Media — estos controllers no tienen cobertura de test **Solución:** Crear controller tests para DashboardController y AuthenticationController

● M3 — Sin validación específica "bus solo ida/vuelta"

Archivo: BookingService.java, TripSegmentService.java **Severidad:** Media — requisito del briefing no implementado completamente **Detalle:** El briefing indica que el desplazamiento entre hoteles no está cubierto. Actualmente no hay validación que impida crear viajes con múltiples destinos intermedios. **Solución:** Añadir validación de que los TripSegments de un viaje solo cubren ida y vuelta

8. Puntos Fuertes ●

Aspecto	Detalle
✓ Seguridad JWT	Secret externalizado, tokens con expiración, roles, rutas públicas mínimas
✓ Motor de precios	4 tipos de descuento (niño, pensionista, grupo, oferta) con endpoint /quote
✓ Email transaccional	Spring Mail + Thymeleaf template + @Async + @TransactionalEventListener
✓ Dashboard directivo	Viajes/año, ganancias año actual, top 3 viajes por facturación
✓ Cloudinary	Upload y delete con configuración externalizada
✓ Validaciones completas	@Valid, @Pattern, @Max, @NotEmpty en todos los DTOs
✓ GlobalExceptionHandler	14 handlers incluyendo catch-all para Exception
✓ 11 controllers con Swagger	Todos tienen @Tag + @Operation
✓ Paginación	Todos los endpoints GET usan Pageable

✓ Soft delete	En todas las entidades principales
✓ Constructor injection	Sin @Autowired en campos
✓ Reglas de negocio	Las 5 reglas críticas implementadas y validadas
✓ DTOs separados	24 DTOs para request/response, sin exponer entidades
✓ Mappers	9 mappers para conversión entidad ↔ DTO

9. Checklist vs Briefing Original

Requisito del Briefing	Estado	Notas
4 entidades (Usuarios, hoteles, autobuses, conductor)	✓	Ampliado a 9 entidades
CRUD para todas las entidades	✓	CRUD completo en 11 controllers
Figma (frontend)	✗ No evaluable	Repositorio solo backend
Draw.io (BBDD + flujo)	✗ No evaluable	Repositorio solo backend
Jira para tareas	✗ No evaluable	Repositorio solo backend
Frontend React conectado	✗ No implementado	Solo backend en este repo
Frontend responsive	✗ No implementado	Solo backend en este repo
Manejo de excepciones	✓	10 excepciones + GlobalExceptionHandler
DTOs	✓	24 DTOs de request/response
Validaciones	✓	Completas en todos los DTOs
Cloudinary	✓	Upload + delete funcional
Viajes en oferta (media/pensión completa)	✓	Endpoint /api/travels/sale + TypeBoard HALF/FULL
Compra múltiples plazas con nombres	✓	Booking acepta múltiples customerIds
Tarifa niño/adulto/pensionista	✓	15% niño, 10% pensionista
Viaje existente o propio	✓	CRUD completo de travels + booking
Email post-compra	✓	Implementado con Thymeleaf template
Vista de usuarios	✓	CRUD + /active
Dashboard directivo	✓	Viajes/año, ganancias, top 3
Descuento IMSERSO/colegio	⚠ Parcial	Descuento grupal genérico (5%, ≥10 pax) no específico IMSERSO

Bus solo ida/vuelta	✗ No implementado	Sin validación de desplazamiento entre hoteles
No reservar si bus/hotel completo	✓	Ambos validados en BookingService
Tests front y back	⚠ Parcial	Solo backend, y tests rotos
No vender viajes pasados	✓	PastTravelException
Menor acompañado de adulto	✓	MinorWithoutTutorException
Conductor 1 bus a la vez	✓	DriverOverlapException

10. Resumen de Archivos Auditados

Archivo	Líneas	Estado
application.properties	34	✓ JWT sin fallback, config completo
pom.xml	142	✓ Spring Boot 4.0.6, Java 25, todas las deps
data.sql	88	✓ Passwords BCrypt
JwtUtil.java	37	✓ English naming, secret via @Value
JwtFilter.java	77	✓ Autenticación, roles, rutas públicas
SecurityConfig.java	19	✓ FilterRegistrationBean en /api/*
GlobalExceptionHandler.java	130	✓ 14 handlers + catch-all
EmployeeController.java	56	✓ DTOs + @Valid + Swagger
TravelService.java	98	✓ Derived queries, paginación, sin shuffle
HotelService.java	104	✓ English naming, reduce/release capacity
BookingService.java	282	✓ Capacity reconciliation, findAllById, minor validation
BookingPricingService.java	237	✓ Descuentos, findAllById
EmailServiceImpl.java	95	✓ Async, Thymeleaf template, event-driven
DashboardService.java	64	✓ Viajes/año, ganancias, top 3
HotelRequestDTO.java	45	✓ @Max(5) en stars
BookingRequestDTO.java	30	✓ @NotEmpty en customerIds
BusRequestDTO.java	25	✓ @Pattern en licensePlate
DriverRequestDTO.java	22	✓ @Pattern en phone
UserRequestDTO.java	34	✓ @Pattern en dni y passport
UserResponseDTO.java	16	✓ Solo @Data, sin redundancias

BusServiceImpl.java	69	✅ BusMapper, sin lógica manual
UserService.java	88	✅ Soft delete
EmployeeService.java	108	✅ @Transactional, BCrypt
TripSegmentServiceTest.java	182	✅ Mock findOverlappingByDriver presente
TravelServiceTest.java	254	❌ Firma sin Pageable
HotelServiceTest.java	240	❌ Firma sin Pageable

11. Prioridad de Acciones

❌ Inmediato (día 1)

1. **Arreglar tests desactualizados:** Actualizar firmas de `getAll()` , `getAvailable()` , `getOnSale()` en 12+ archivos de test para usar `Pageable`

🟡 Corto plazo (día 2-3)

2. Actualizar README: corregir `/activos` → `/active` , `/disponibles` → `/available`
3. Quitar `+` unario sobrante en `BookingPricingService.java:41`
4. Añadir tests para `DashboardController` y `AuthenticationController`
5. Añadir `@ApiResponse`s y `@Schema` a todos los endpoints

🟠 Medio plazo (sprint)

6. Añadir manejo de errores en `CloudinaryService` (timeout, tamaño)
7. Implementar validación "bus solo ida/vuelta" (desplazamiento entre hoteles no cubierto)
8. Hacer `data.sql` resistente a IDs existentes
9. Versionado de API (`/api/v1/`)
10. Añadir `@DataJpaTest` para repos, tests de integración con H2, tests de seguridad JWT

12. Conclusión

El proyecto tiene una **base muy sólida**. Prácticamente todos los issues críticos y altos de la auditoría anterior han sido corregidos:

- ✅ Seguridad JWT correcta (sin fallbacks hardcodeados, sin rutas whitelisteadas)
- ✅ `EmployeeController` con DTOs y validaciones
- ✅ Catch-all Exception handler
- ✅ `TravelService` con queries derivadas y paginación (sin regresiones)
- ✅ Validaciones completas en todos los DTOs
- ✅ Soft delete consistente
- ✅ English naming en todo el código
- ✅ Swagger en todos los controllers

El **único problema crítico** que persiste son los **tests desactualizados** que no compilan tras la refactorización a `Pageable` . Una vez corregido, el proyecto estaría en excelente estado para producción.

Puntuación actual: 8.5/10 (con tests rotos) **Puntuación potencial: 9.5/10** (con tests corregidos y mejoras menores)